

# The Sidecar Pattern Explained: Decoupling Operational Features



BYTEBYTEGO

DEC 19, 2024



208



10

Share

Design patterns are reusable solutions to common problems in software design. They provide a structured approach to solving architectural challenges without reinventing the wheel each time.

The sidecar pattern is one such design pattern that has gained prominence in modern software engineering.

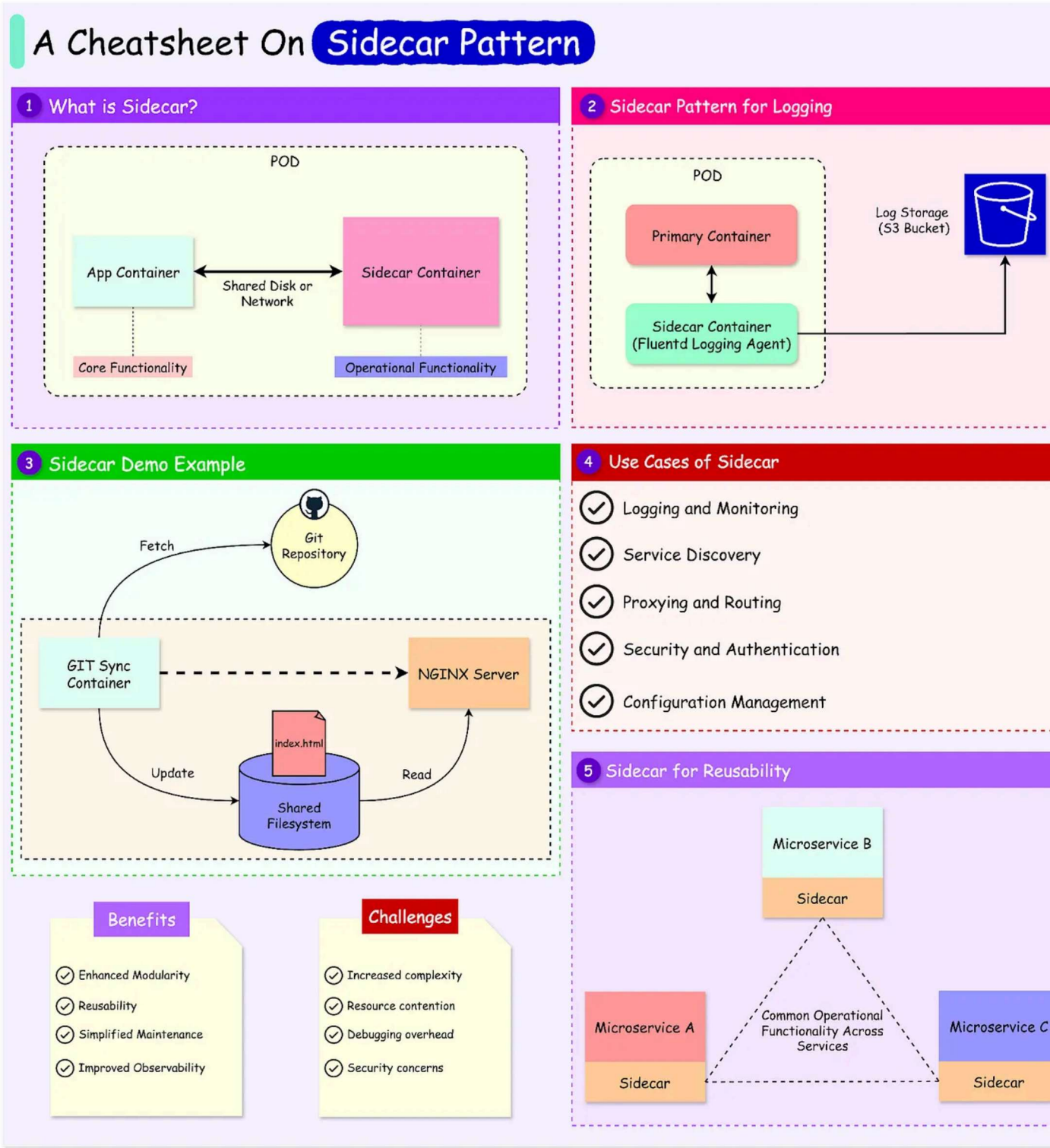
At its core, the sidecar pattern pairs a secondary process or service (the "sidecar") with a primary application to handle complementary tasks. These tasks include logging, monitoring, proxying, security, or configuration management. The sidecar runs alongside the main application, sharing the same host or container, but remains logically and operationally independent.

The sidecar pattern can be compared to a motorcycle with a sidecar. The motorcycle (the primary service) is the main driver, responsible for the core functionality, like transporting a person. The sidecar (the auxiliary service) carries additional tools or passengers, assisting the main vehicle without interfering with its operation.

Similarly, in software systems, the sidecar extends the capabilities of the primary application without being tightly coupled to it.

In this article, we'll learn about the sidecar pattern in detail and understand how it works. In the end, we will also look at its benefits and challenges that can help us

make better decisions when using the pattern.

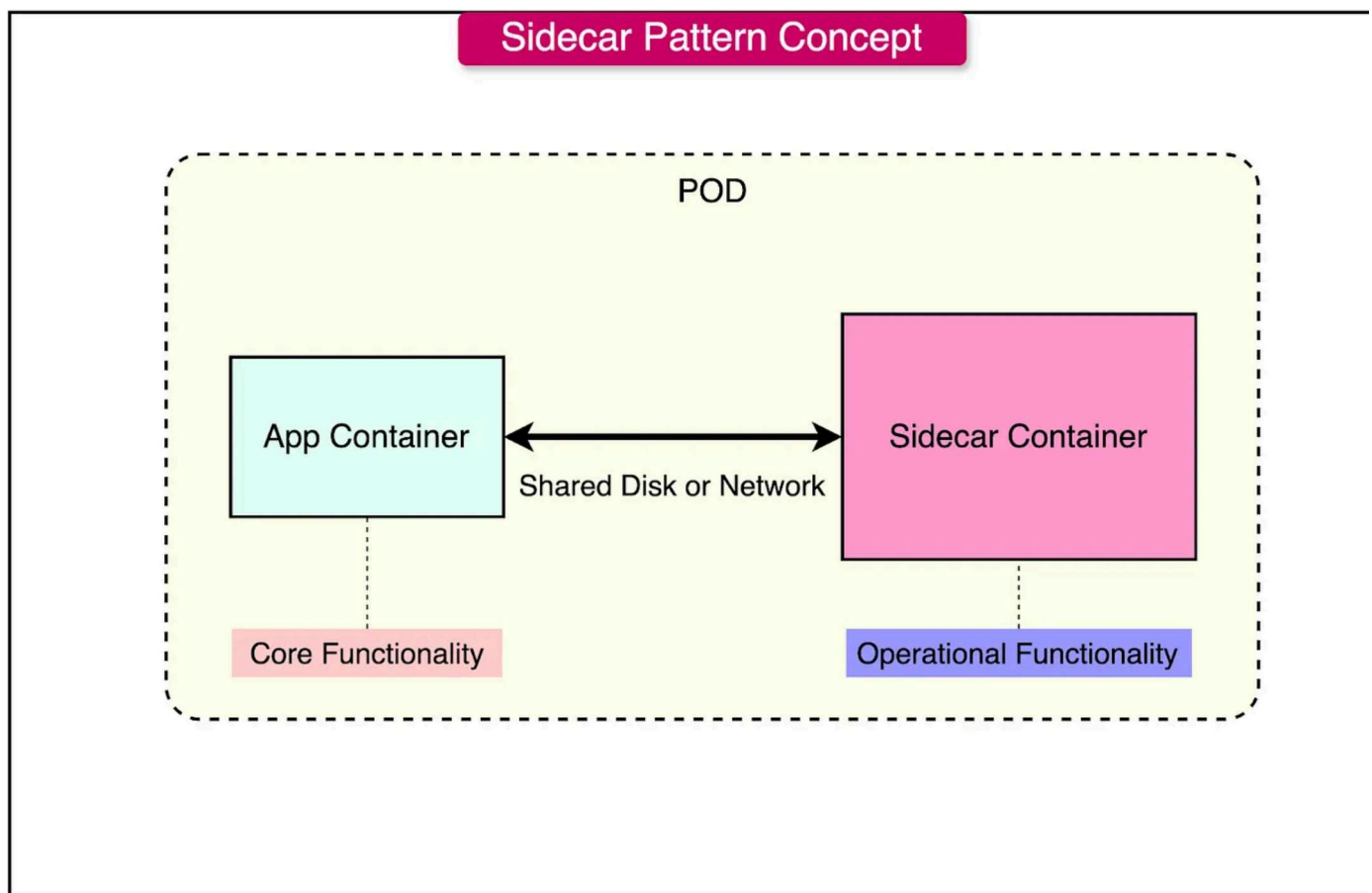


# Core Concept of the Sidecar Pattern

The sidecar pattern is a structural design pattern used in distributed systems architecture.

It involves pairing a primary application or service with a secondary, co-located process known as the sidecar.

See the diagram below that shows the concept of a bare-minimum sidecar implementation using a Kubernetes Pod.



There are two containers in this setup:

- The application container has the core logic that serves the business requirements.
- The sidecar container augments the application container.

There are some key characteristics associated with the pattern:

- **Co-location:** The sidecar is deployed alongside the primary service, often in the same pod in Kubernetes. This ensures low-latency communication and shared resource access.
- **Independence:** The sidecar is loosely coupled to the main application, allowing for independent updates and debugging.
- **Resource Sharing:** Both services share resources such as CPU, memory, and network interfaces.
- **Communication:** Communication between the primary service and the sidecar is typically facilitated via local network interfaces or shared volumes.

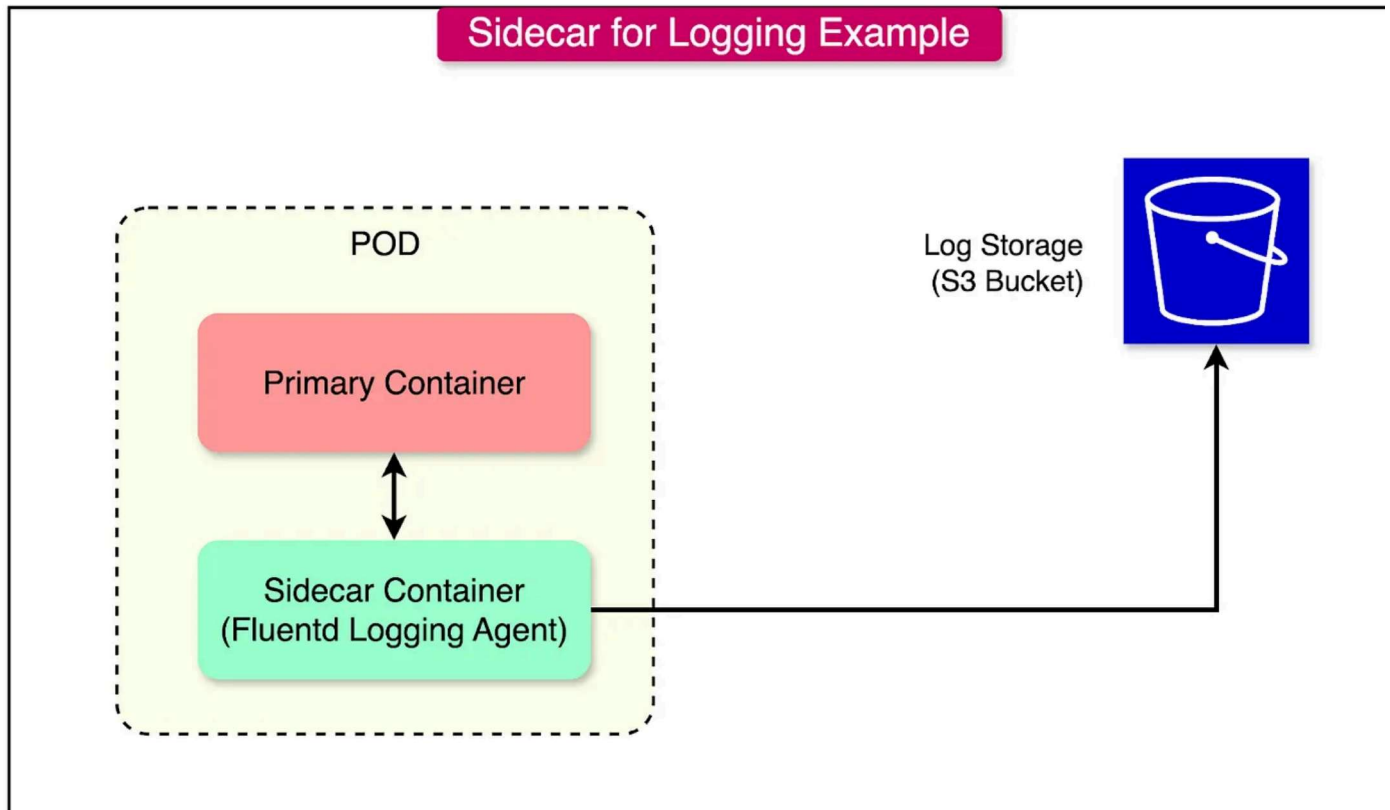
## Common Use Cases of Sidecar

The sidecar pattern is well-suited for distributed systems and containerized environments. Below are some of the common use cases:

- **Logging and Monitoring:** Sidecars like Fluentd or Logstash can aggregate logs from the main application and forward them to a centralized logging system. This approach decouples log processing from application logic.
- **Service Discovery:** Sidecars act as service discovery agents, dynamically registering the primary service with a service registry (for example, Consul, etc). This ensures that the service can locate and communicate with other services in the system without embedding discovery logic in the application.
- **Proxying and Routing:** Proxies like Envoy or HAProxy are used as sidecars to handle incoming and outgoing network traffic. They provide features like load balancing, circuit breaking, and request/response transformation, which can be managed independently of the main application.
- **Security and Authentication:** Sidecars can handle security concerns, such as injecting TLS certificates (for example, using HashiCorp Vault) or managing API tokens. This centralizes security practices and reduces the burden on application developers.

- **Configuration Management:** Sidecars can periodically fetch or reload configuration settings from a central repository, ensuring that the primary service operates with the latest configurations without requiring restarts.

See the diagram below that shows the use of a sidecar container in the context of logging:



## Architectural Components of the Sidecar Pattern

From an architectural point of view, the sidecar pattern consists of two primary components and a mechanism for their communication:

- **Primary Service:** The main application or service for core business logic and functionalities. It is designed to operate independently of the auxiliary tasks managed by the sidecar. For example, a web application, an API service, or a database.

- **Sidecar Service:** A lightweight, co-located service that handles auxiliary tasks such as logging, monitoring, service discovery, or security. The sidecar focuses non-core functionalities to support the primary service. For example, Fluentd for logging, Envoy Proxy for traffic routing, or Vault Agent for managing secrets.
- **Communication Mechanisms:** These define how the primary service and sidecar interact and share data. Communication is typically designed to be efficient and localized within the same environment.

## Sidecar Deployment in Containerized Environments

In containerized environments, the sidecar pattern is implemented as part of a pod.

A pod is the smallest deployable unit in Kubernetes and can contain multiple containers. The primary and sidecar containers are deployed within the same pod, which means they share resources and a network namespace.

- **Shared Network:** Both containers use the same IP address and communicate via localhost or specific ports.
- **Shared Volumes:** Kubernetes supports shared volumes within a pod, allowing both containers to access the same filesystem for data exchange.

Here's an example of a Kubernetes YAML file for a Pod that demonstrates the sidecar pattern:

```
apiVersion: v1
kind: Pod
metadata:
  name: sidecar-example
spec:
  containers:
    - name: primary-service
      image: my-primary-service:latest
    ports:
```

```
- containerPort: 8080
volumeMounts:
- name: shared-volume
  mountPath: /shared
- name: sidecar-service
  image: my-sidecar-service:latest
  volumeMounts:
- name: shared-volume
  mountPath: /shared
volumes:
- name: shared-volume
  emptyDir: {}
```

The containers section in the YAML lists two containers that will run within the Pod.

- First, we have the primary service that runs an image named `my-primary-service` using the latest version. It exposes port 8080 for communication with this container and mounts a shared volume at the directory `/shared` inside the container.
- The second container is the sidecar service. The container runs an image named `my-sidecar-service` using its latest version. It mounts the same shared volume at the `/shared` directory, enabling data sharing between the sidecar and primary service.

The volume is defined later using `emptyDir`. It creates a temporary empty directory that exists only while the Pod is running. This allows the two containers to share files.

## Sidecar Pattern Example Demo

Now that we've covered the basics of the sidecar pattern and its applications, let's dive into a practical demonstration of the pattern using Kubernetes.

In this example, we'll use the sidecar pattern to implement a basic Git-based workflow for deploying new code to a running service.

The Kubernetes pod will include two containers:

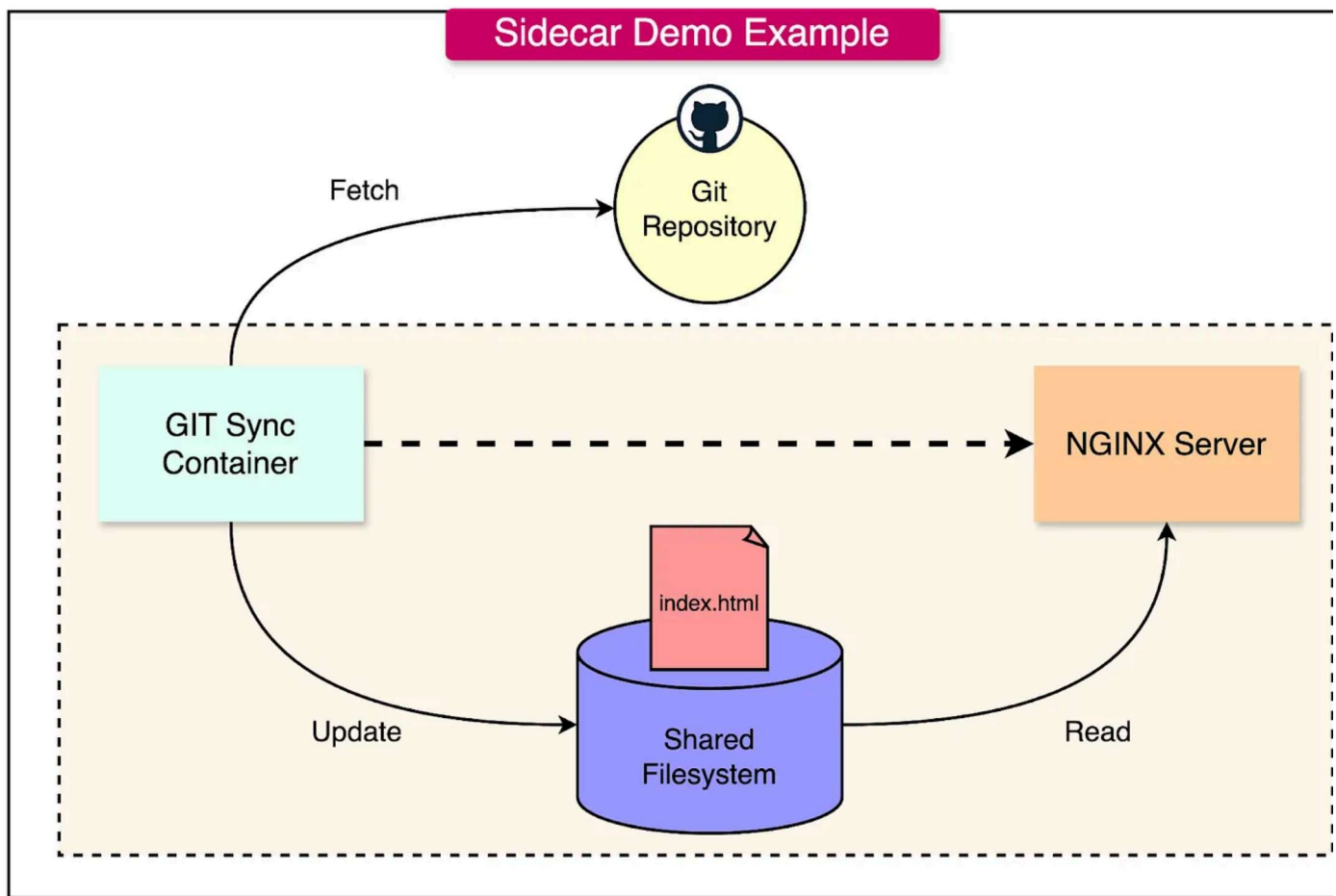
- **Nginx container:** Serves the index.html file stored in a shared filesystem.
- **Sidecar container:** Continuously polls a Git repository for the latest version of index.html and updates it in the shared filesystem.

The workflow will be as follows:

- When changes are pushed to the Git repository, the sidecar container retrieves updated index.html file and writes it to the shared filesystem.
- The Nginx container automatically serves the updated file, ensuring the latest changes are reflected without requiring a pod restart.

The diagram below shows the various parts of this demo. Note that this is just an example and the code is just to demonstrate the concept of sidecar pattern in action





## Step 1: Deploying the Nginx Container

We begin by creating a simple HTML file to be served by the Nginx server.

```
<html>
  <body bgcolor="#FFFFFF">
    <h1><strong>This is demo for the Sidecar Pattern</strong></h1>
  </body>
</html>
```

Next, we build a Docker image to deploy this file using Nginx. Below is a basic Dockerfile for this purpose.

```
FROM nginx
```

```
ADD index.html /usr/share/nginx/html/index.html
```

Finally, we create a Kubernetes deployment for Nginx:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: nginx
    name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - image: demo-example/nginx
          name: nginx
          ports:
            - containerPort: 80
```

## Step 2: Implement the Sidecar Container

Next, we build a Docker image for the sidecar container that runs a synchronization script named “page-refresher.sh”. This script polls a Git repository every few seconds for updates to the HTML file.

```
FROM alpine
```

```
WORKDIR /usr/share/nginx
```

```
ADD page-refresher.sh page-refresher.sh
```

```
ENTRYPOINT sh page-refresher.sh
```

We build the image and update the Kubernetes deployment to use it as a sidecar container.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: nginx
  name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - image: demo-example/sidecar
          name: sidecar
          env:
            - name: STATIC_SOURCE
              value: https://raw.githubusercontent.com/your-repo/sidecar-demo/main/index.html
          volumeMounts:
            - name: shared-data
```

```
    mountPath: /usr/share/nginx/html
- image: demo-example/nginx
  name: nginx
  ports:
    - containerPort: 80
  volumeMounts:
    - name: shared-data
      mountPath: /usr/share/nginx/html/
  volumes:
    - name: shared-data
      emptyDir: {}
```

The key points to note here are as follows:

- **Sidecar Container:** The demo-example/sidecar image runs the synchronization script.
- **Shared Volume:** Both containers share the emptyDir volume, ensuring that updates from the sidecar are visible to the Nginx container.
- **Environment Variable:** The STATIC\_SOURCE variable specifies the Git repository URL where index.html is stored.

## Step 3: Testing the Workflow

The workflow can be tested as follows:

- Modify the index.html file in the Git repository and push the changes.
- The sidecar container polls the repository, retrieves the updated file, and writes it to the shared volume.
- The Nginx server automatically serves the updated file from the shared filesystem.
- Export a NodePort Kubernetes service to access the application and view the contents of the HTML file in the browser.

# Advantages of Using the Sidecar Pattern

The sidecar pattern offers several advantages making it a useful choice in modern distributed systems.

Here's a detailed look at these benefits with practical examples:

## 1 - Modularity

By isolating non-core functionalities into a separate sidecar service, the sidecar pattern promotes modularity. The primary service focuses solely on its business logic while the sidecar handles auxiliary tasks like logging, monitoring, or routing.

For example, a web application can delegate traffic routing and load balancing to a sidecar proxy like Envoy. This separation ensures that any updates to routing rules or traffic policies do not require changes to the web application's codebase. If an issue arises in the routing logic, it can be fixed in the sidecar without affecting the primary application's functionality.

## 2 - Simplified Maintenance

Sidecars can be updated, restarted, or replaced independently of the primary application.

This simplifies the maintenance process and reduces downtime, as auxiliary tasks can evolve without disrupting the core service.

For example, a sidecar handling TLS certificate renewal (e.g., using Certbot) can be updated to support new cryptographic standards without requiring changes to the primary web server.

## 3 - Improved Observability

Sidecars play a crucial role in improving observability by centralizing logging, metrics collection, and tracing.

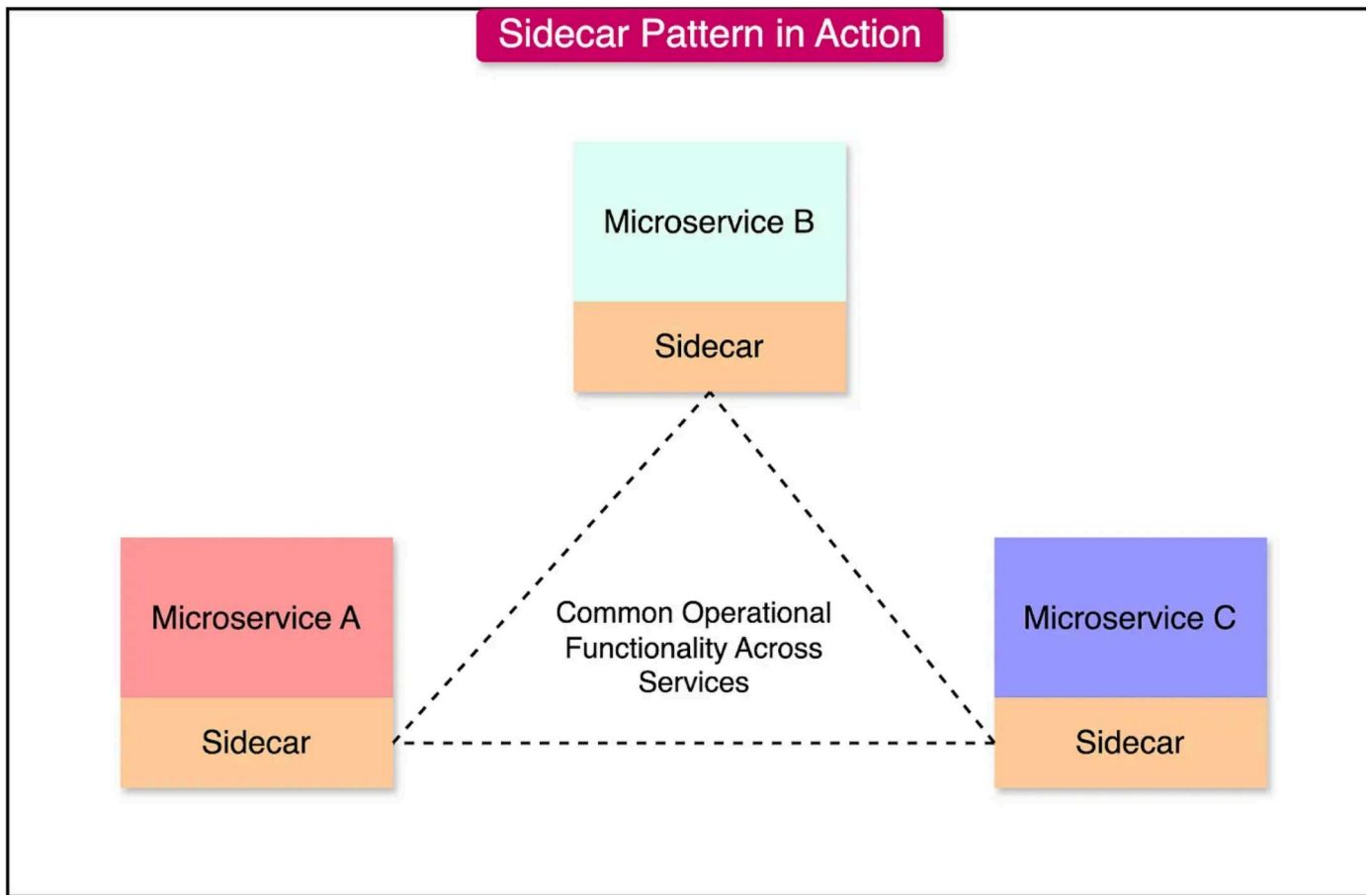
For example, a metrics collection sidecar runs alongside the main application, collecting data such as CPU usage, memory consumption, and request rates. This data is forwarded to a monitoring dashboard (such as Grafana) for visualization.

## **4 - Reusability**

A single sidecar implementation can be reused across multiple services, standardizing certain functionalities across a distributed system. This reduces development effort and promotes consistency.

For example, a sidecar for service discovery can be deployed alongside all microservices in a system, ensuring that each service can dynamically locate and communicate with others.

See the diagram below that shows multiple microservices sharing a single sidecar:



## Challenges and Trade-offs with the Sidecar Pattern

While the sidecar pattern offers some great advantages in modularity, scalability, and maintainability, it is not without challenges.

Below are the key challenges and strategies to mitigate them:

### 1 - Increased Complexity

The sidecar pattern adds a new layer of architectural complexity by introducing additional components and interactions.

Managing the lifecycle, deployment, and communication between the primary service and sidecar can become challenging, particularly in large-scale distributed systems.

the number of services grows, debugging and troubleshooting inter-container dependencies becomes increasingly difficult.

To mitigate this, we can use tools like Helm to simplify the configuration and management of sidecars in Kubernetes. Also, it helps when there is a clear separation of responsibilities between the primary service and the sidecar.

## 2 - Resource Contention

Since the primary service and sidecar share the same environment, they compete for resources such as CPU, memory, and disk I/O. Poorly optimized sidecars can negatively impact the performance of the main application.

For example, a logging sidecar like Fluentd performing high-volume log aggregation may consume excessive CPU or memory, leading to degraded application performance.

To mitigate this, we can set resource limits and requests in Kubernetes pod configurations to ensure fair resource allocation. We can also optimize sidecar configurations to reduce their resource footprint.

```
resources:
  requests:
    memory: "256Mi"
    cpu: "500m"
  limits:
    memory: "512Mi"
    cpu: "1"
```

## 3 - Debugging Overhead

Sidecar-based architectures increase the number of components involved in a system, making debugging more complex. When an issue arises, it can be challenging to



determine whether the problem originates in the primary service, the sidecar, or the interaction.

For example, a failure in a sidecar service can disrupt communication for the entire application, leading to cascading failures.

To mitigate this, it's important to implement logging and monitoring for both primary service and sidecar to get clear visibility. Also, it can be a good idea to adopt a circuit breaker pattern for sidecar dependencies to minimize the impact of sidecar failures.

## 4 - Security Concerns

Sidecars can become a security vulnerability if not properly secured.

A misconfigured sidecar might expose sensitive data, such as logs containing personal information to unauthorized users. Also, vulnerabilities in sidecar dependencies can be exploited to compromise the system.

As a mitigation approach, it's important to follow best practices for securing containerized applications and using appropriate network policies. Also, regularly audit sidecar configurations and perform security scans.

## Summary

In this article, we've taken a detailed look at the sidecar pattern along with a basic implementation example.

Let's summarize our learnings in brief:

- The sidecar pattern is a design pattern commonly used in distributed systems to decouple auxiliary tasks from the main application. It is used to enhance modularity and reusability.

- The pattern involves deploying a sidecar service along with the primary application in the same environment so it can share resources.
- The primary and sidecar services communicate via shared volumes or network interfaces.
- The sidecar pattern can be implemented using Kubernetes pod. This was demonstrated through an example of a Git-based workflow where a sidecar synchronizes content from a Git repository to a shared filesystem.
- The benefits of sidecars include enhanced modularity, maintenance, observability and reusability.
- The challenges of a sidecar are increased complexity, resource contention, debugging overhead, and possible security concerns.
- Sidecars are used in scenarios like service discovery, logging, monitoring, and dynamic configuration management.

---

## Subscribe to ByteByteGo Newsletter

Explain complex systems with simple terms, from the authors of the best-selling system design book series  
Join over 1,000,000 friendly readers.

Upgrade to paid



208 Likes · 10 Restacks

## Discussion about this post

Comments   Restacks



Write a comment...